>_ OpenAB

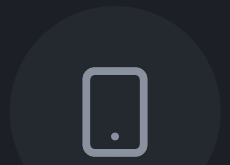
讓 Agentic Coding 更接近解放 Terminal 的工作模式

如何把 Coding Agent 從 Terminal 帶進真正隨時協作的工程流程

Neo Hsu

我要解放的，不只是輸入指令的地方

不只是



在手機上
輸入 Terminal 指令

而是

- 隨時交代工作
- 過程可見、工作可接續

去年：自己寫 adapter，把聊天接到 CLI

CHAT ENTRY

聊天入口

Telegram
Discord

MY ADAPTER

我的 adapter

我自己維護的接線

CODING CLI

Coding CLI

Headless / -p

› 可以丟任務、看結果，再繼續追問。

聊天室是連續的，呼叫卻是一輪一輪



! 這是我當時的 one-shot 接法，不是所有 headless 的限制。

要接續工作，adapter 就得懂得越來越多



› 接得通不難；接成同一項持續的工作，才是負擔。

第一個難題：工作對話怎麼延續？

01
延續

02
互動

03
例外



Session lifecycle

怎麼建立、接續、結束？



Context 邊界

哪些交給 Agent 管理？

› 先把工作對話與上下文的責任分清楚。

第二個難題：執行中的互動怎麼表達？

01
延續

02
互動

03
例外



Streaming

過程如何即時回來？



Tool / permission

事件與決策怎麼表達？

› 互動不該全部被壓成最後的一段 stdout。

第三個難題：中斷、恢復、失敗怎麼接回來？

01
延續

02
互動

03
例外

Cancel / resume

停止之後，如何再接續？

Error handling

失敗應回到哪段工作？

› 換一個 CLI，這些接法又可能要重新整理。

我需要的，是共同的互動方式



› 共同介面，讓入口與後端能分開演進。

難的不是呼叫 API，而是整理行為差異

01	**能力範圍**	有哪些可用能力？
02	**事件與時機**	什麼時候能呼叫？
03	**工具與權限**	怎麼執行？何時要求決策？

› 呼叫可以很快接通，行為的一致性仍要整理。

不重做 Agent，工程化它工作的環境

YOU DESIGN

我來設計工作環境

入口、執行環境與協作方式

入口

執行環境

協作方式

REUSE

沿用既有 Agent

各家的工具、能力
與 context 管理

› 不是 SDK 做不到，而是我的整合與維護取捨。

ACP：把持續互動納入協定



› 不是另一個 Agent，也不是單純多包一層 CLI。

同一個 session，可以有多輪工作互動



› 初始化與能力協商後，再進入工作對話。

Context 留在 Agent runtime，入口管理工作對應



› 共同協定，不會自動搬移不同 Agent 的內部記憶。

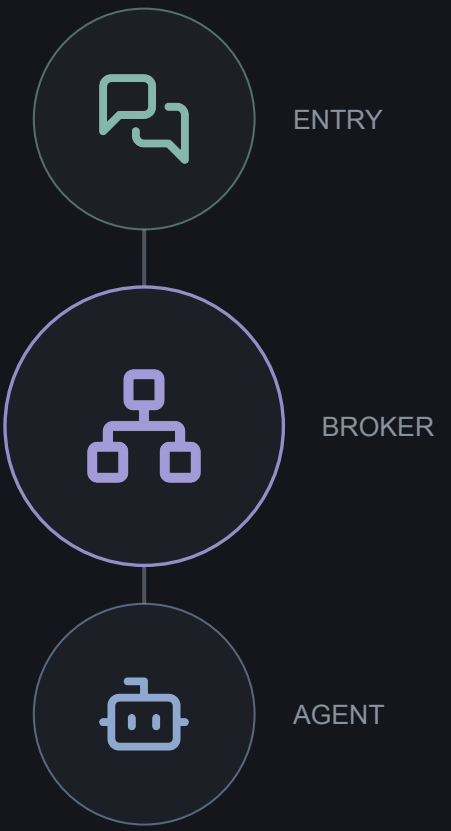
今年四月 / 我的發現與嘗試

OpenAB

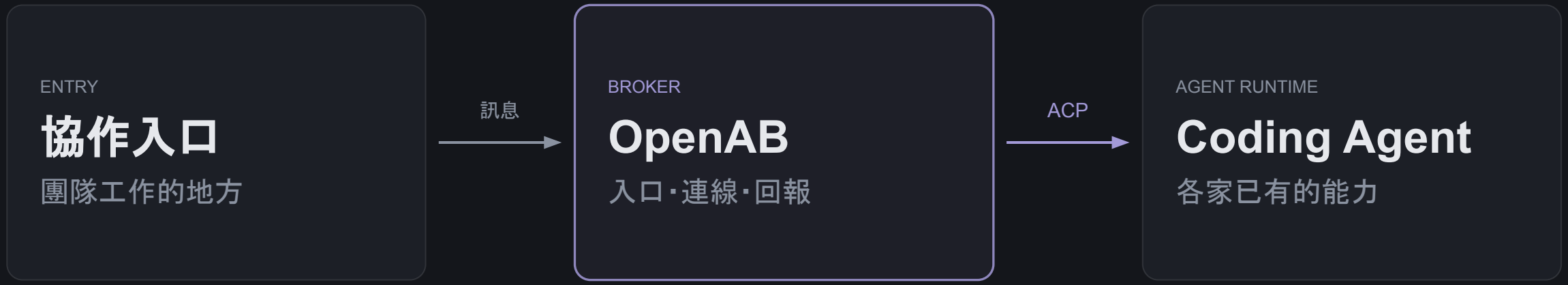
Open Agent Broker

把共同協定，放進實際協作與執行環境。

外部開源專案 github.com/openabdev/openab



OpenAB 不只接聊天，也管理工作如何運行



› Open Agent Broker：入口、連線與 Agent 之間的橋接層。

入口可以是 Discord、Slack 或 Custom Gateway



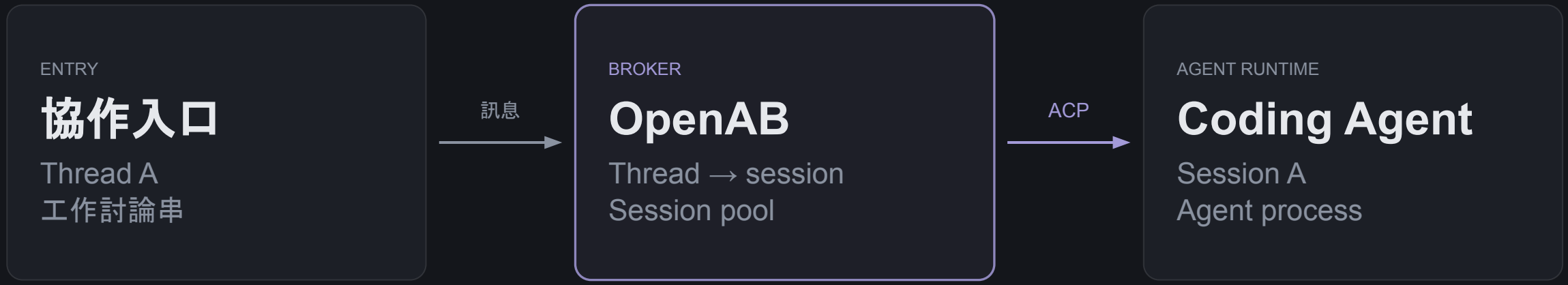
› 進入既有協作環境；各平台能力仍可能不同。

後端保留各家的 Coding Agent



› 版本、登入、工作目錄與 ACP 接法仍要配置。

Thread 對上 session，pool 管理工作連線



› 建立、重用、回收工作連線；不等於隔離檔案。

過程回得來，不只最後一段文字



! 有權限協定，不等於聊天室每次都等待人工批准。

能取消、能回報錯誤；恢復則有條件



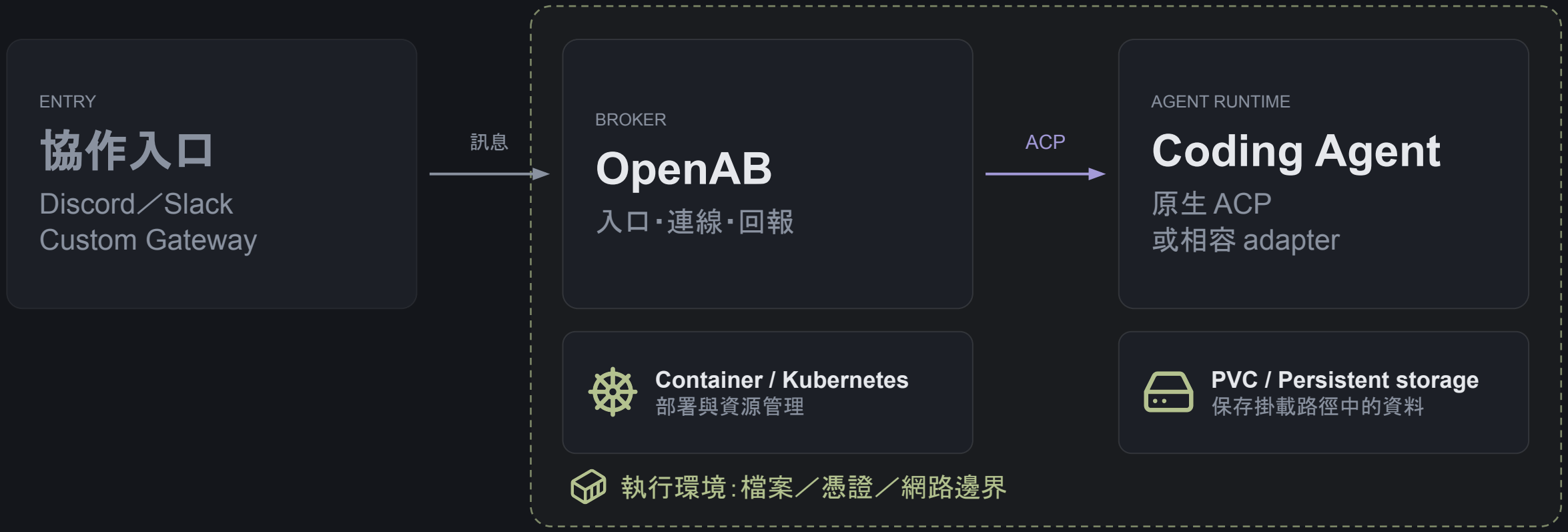
! 恢復可能失敗，不保證永遠保有完整記憶。

讓 Agent 有明確的執行環境



› Container / sandbox 策略，是需要配置的邊界。

需要時，再加入 Kubernetes 與持久化



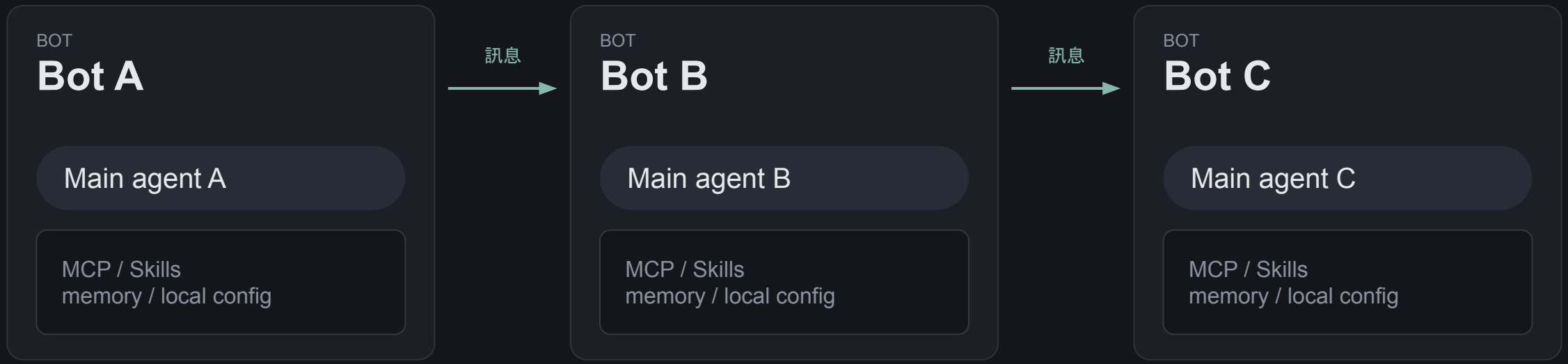
! 保存資料 ≠ 自動恢復對話；也不必一開始就上 Kubernetes。

管理、恢復、隔離，是三件不同的事

可管理	OpenAB	入口與工作連線
條件式恢復	Agent runtime	能力與保存狀態
依配置隔離	執行環境	檔案、憑證、網路

› 這是運行基底，不是「裝好就全部成立」的保證。

Bot-to-bot messaging，讓交接進到協作環境



依團隊的交接指示點名，接收端也必須允許

› 各 Bot 對應自己的 workflow，再透過訊息交接。

Thread session：讓訊息有工作歸屬



› 同串可協作，各 Bot 仍有自己的 session 與狀態。

Allowlist 與 turn cap：界定訊息互動邊界



Allowlist：誰可以參與

Turn cap：連續 Bot 回合上限

! 降低互相叫醒的風險，不是完整的流程或安全保證。

OpenAB 提供協作零件，不替你決定流程

PROVIDED BY OPENAB

OpenAB 的協作零件

- 訊息
- thread
- 來源限制
- 回合上限
- 回覆關聯

DESIGNED BY YOUR TEAM

團隊的流程規則

- 誰接手？
- 失敗怎麼辦？
- 何時算完成？
- 何時要人決定？

! 不是開啟 bot-to-bot，就得到完整的 workflow orchestrator。

少寫部分串接 腳本，不是省掉所有流程責任

可以交給 OpenAB

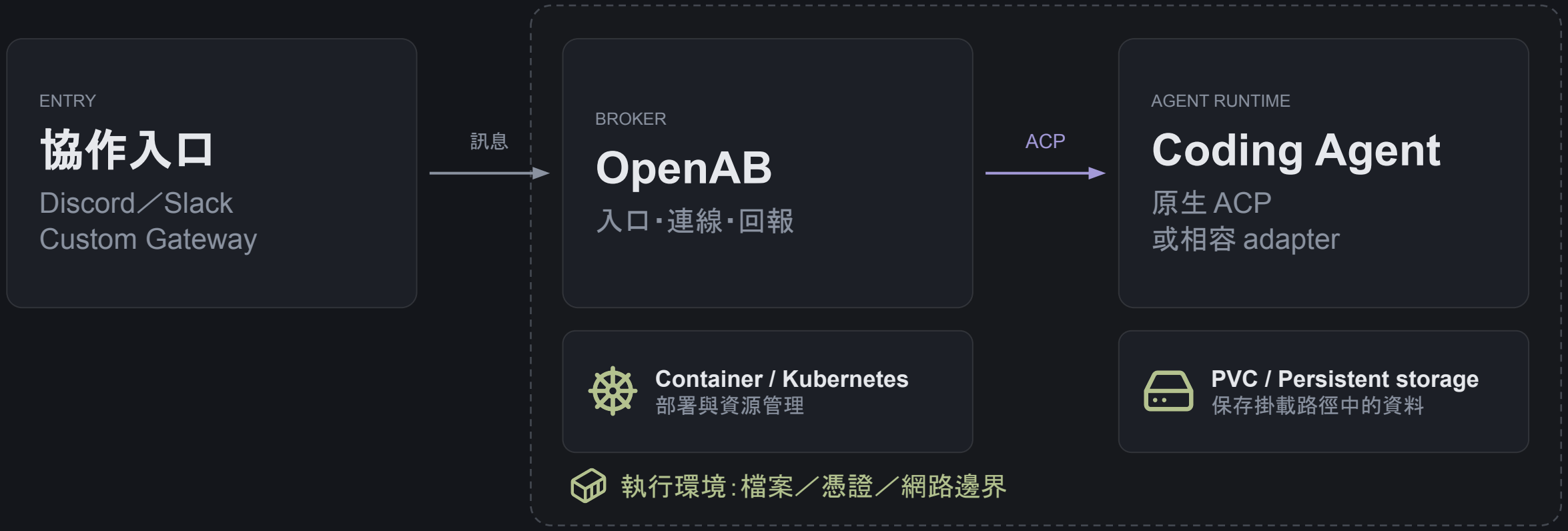
部分訊息轉送、對話連線與回報接線

仍需另外設計

可靠重試、去重
交付規則與高風險控制

› 把可共用的接線交給工具，把工程責任留清楚。

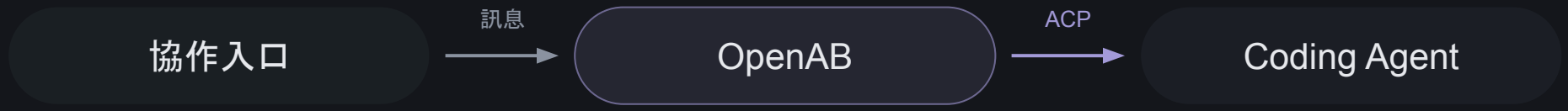
從聊天入口，走向可接續的工程工作



› 可管理 ／ 條件式恢復 ／ 依配置隔離 ／ 由規則串接

ACP：把持續互動納入協定
OpenAB：把協定放進協作與執行環境

不重做每個 Agent，
用共同協定把它接進團隊的協作與執行環境。



Q&A

問題與討論 ／ 3 分鐘

Neo Hsu

OpenAB github.com/openabdev/openab

ACP agentclientprotocol.com

Headless 不等於沒有 session 或 streaming

CLI 仍可能支援

- ✓ 續聊與指定 session
- ✓ 結構化輸出與串流

我的整合痛點

- ⚠ 跨 CLI 的接法與行為
- ⚠ 仍需要整理與維護

Claude Code：-p + --continue / --resume / --output-format stream-json

› SDK 也能重用 Agent 能力；這是取捨，不是能力二分。

ACP：方法名稱與能力邊界

initialize	版本與能力協商
session/new	建立工作對話
session/prompt / session/update	提交一輪 / 回報事件
session/request_permission	向 client 要求權限回應
session/cancel	要求中止，不是回滾
session/load / session/resume	依版本與能力宣告；語意不同

› ACP v1 參考；OpenAB 所查核心 client 以 session/load 嘗試恢復。

OpenAB 的能力，要連同條件一起 說

權限	所查 ACP client 自動回應，不是每次人工批准。
恢復	要有保存狀態與後端支援；可能失敗或新開。
隔離	Session pool 不等於獨立檔案系統或 sandbox。
Bot 回合	Turn cap 受配置影響；不是可靠交付保證。
定向回覆	Discord／Feishu 有實作；Slack 行為不同。

! 版本、平台、Agent、設定一起核對，不只看功能名稱。

Workflow handoff，先定義這五件事

1

INPUT

接手者需要的
任務、限制
與上下文

2

ARTIFACT

成果放在哪裡，
接手者是否
拿得到

3

VERSION

如何識別這次
交接的版本

4

READY

達到什麼條件
才交接，交給誰

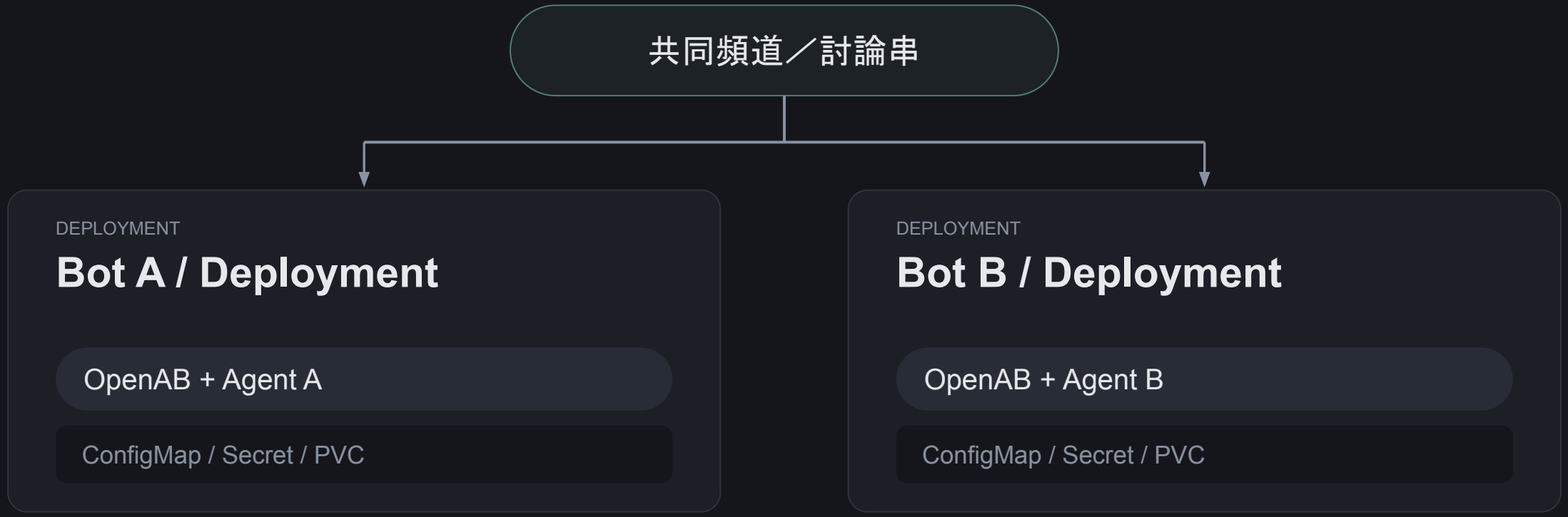
5

FAILURE

失敗、重試、
停止與人類
介入責任

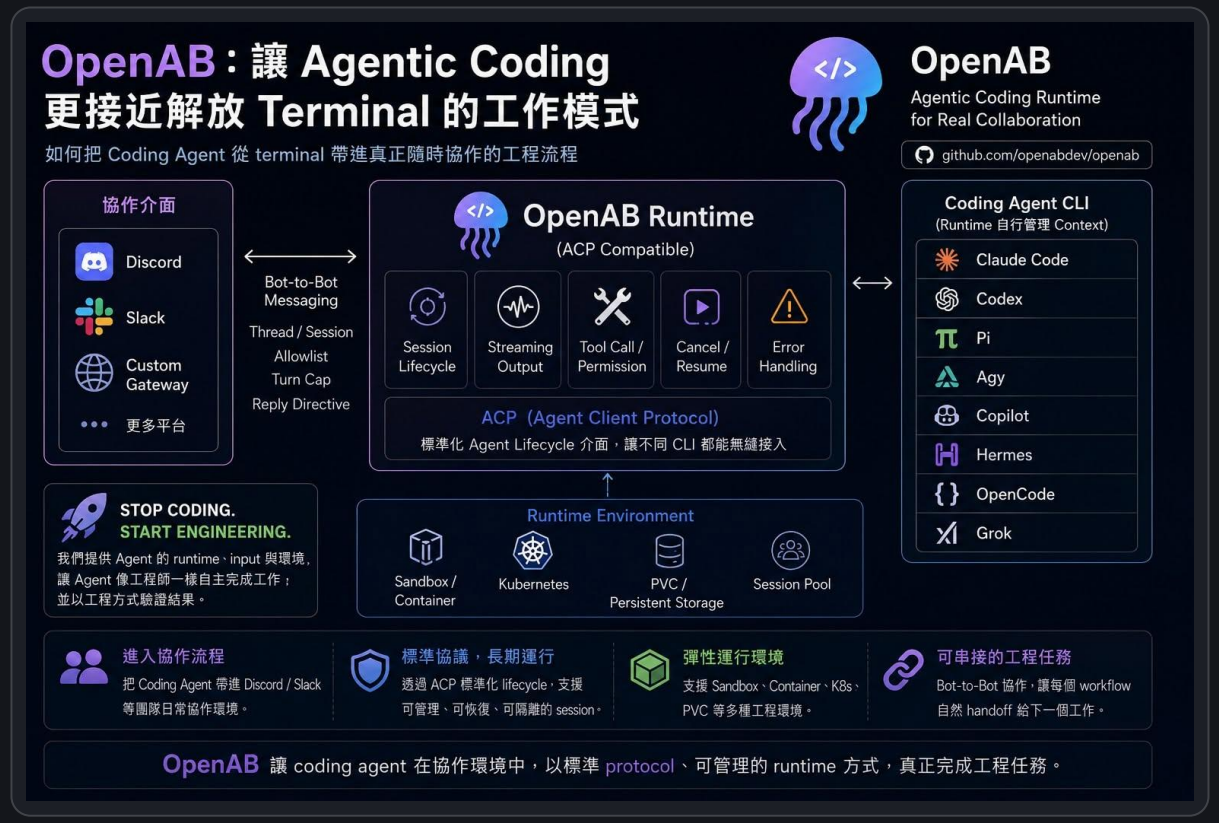
› 這是團隊的設計清單，不是 OpenAB 內建工作流格式。

多 Bot 部署，不等於共享所有狀態



› Helm 多 Agent 模式：各自的 Pod 與狀態，成果存取另設計。

使用者提供的概念總覽



! 使用者提供的概念原圖：相容、恢復與隔離皆有條件，不作功能驗證證據。